BUG-013 缺陷报告：下单金额可被篡改（Fiddler 断点测试）

2026-10-21 · 韩锟谊 · ShopEase 接口测试

缺陷概要

用 Fiddler 请求前断点把下单请求里的单价和总价改成 0.01 后，后端照单全收：两台原价 $29,998 的 MacBook Pro 以 $0.01 下单成功（订单 #24）。修复后后端按数据库价格计算，同样的篡改下单金额为 $29,998，8 条回归用例全部通过。

项	内容
编号	BUG-013
标题	下单接口未校验价格，篡改请求中的金额后可用 0.01 元下单
模块 / 接口	订单，`POST /shopease/api/orders.php`（创建订单）
严重程度	高：直接造成资金损失
优先级	高
发现方式	抓包工具 Fiddler Classic 的请求前断点（`bpu orders.php`），拦截请求后手工篡改再放行
来源	测试报告已知风险第 2 条"订单金额由前端传入、后端不重新计算"，原为看代码推测，本次已实际复现
状态	已修复，回归通过（2026-10-21）

测试环境与工具

项	内容
操作系统	Windows
服务端	XAMPP：Apache 2.4.58、PHP 8.2.12、MariaDB 10.4.32
浏览器	Edge
抓包工具	Fiddler Classic（免费经典版）
访问地址	`http://localhost.fiddler/shopease/`（浏览器访问 localhost 不走代理，换成这个地址 Fiddler 才能抓到）
测试账号	`xss1@test.com`，普通用户，id 为 17
测试商品	MacBook Pro 14-inch M3 Chip，商品 id 2，单价 $14,999

测试方法：Fiddler 请求前断点

页面上的价格不能直接修改，所以要在请求离开浏览器之后、到达服务器之前改它。Fiddler 作为代理夹在浏览器和服务器中间，请求前断点能把请求扣下，改完再放行，用来验证后端有没有自己校验数据。

断点类型	拦在哪里	用来测	本次是否使用
请求前断点（`bpu`）	请求发给服务器之前	后端校验	是
响应后断点（`bpafter`）	响应交给浏览器之前	前端异常处理	否

准备工作：

1. 打开 Fiddler，确认 File 菜单里 Capture Traffic 已勾选。
2. Filters 标签勾选 Use Filters，Hosts 选 Show only the following Hosts，填 `localhost; localhost.fiddler`，点 Actions → Run Filterset now，只显示 ShopEase 的请求。
3. 浏览器用 `http://localhost.fiddler/shopease/` 打开网站并登录测试账号。

复现步骤与结果

1. 以测试账号登录，把 MacBook Pro 14-inch M3 Chip 加入购物车，数量 2，进入结算页，填好收货信息，先不点下单。
2. 在 Fiddler 左下角 QuickExec 输入 `bpu orders.php`，回车，设置请求前断点。
3. 点"下单"。浏览器显示 Processing...，Fiddler 里出现一行带红色图标的 `/shopease/api/orders.php`，表示请求已被扣住。
4. 单击这一行，Inspectors 上半部分选 TextView，把 `"price":14999` 改为 `"price":0.01`，把 `"total_price":29998` 改为 `"total_price":0.01`，其余内容不动。
5. 点红色提示栏里的 Run to Completion 放行。
6. QuickExec 输入 `bpu` 回车取消断点，点工具栏 Go 放行其他被扣住的请求。
7. 打开 Orders 页面查看新订单。

字段	原值	篡改值	预期结果	实际结果
商品单价 `price`	14999	0.01	按数据库价格 $14,999 记录	记录为 $0.01，商品行显示 $0.02（0.01 × 2）
订单总价 `total_price`	29998	0.01	按数据库计算为 $29,998，或拒绝下单	订单 #24 创建成功，总金额 $0.01，状态 Processing

商品行 $0.02 和总金额 $0.01 对不上，说明后端既没有按数据库价格重算，也没有检查"总价 = 单价 × 数量"。

证据：Fiddler 中被拦截请求的 TextView 截图（显示修改后的 price 和 total_price）；Orders 页面订单 #24 截图（显示 Total Amount $0.01）。

原因分析

`api/orders.php` 创建订单时，总价、商品单价、商品名都直接取自前端请求，写入数据库前没有和 products 表核对。

```
$total = (float)($data['total_price'] ?? 0);
```

```
foreach ($items as $item) {
    $pid   = (int)($item['id']       ?? 0);
    $pname = $item['name']           ?? 'Unknown';
    $qty   = (int)($item['quantity'] ?? 1);
    $price = (float)($item['price']  ?? 0);
    $itemStmt->bind_param("iisid", $orderId, $pid, $pname, $qty, $price);
    $itemStmt->execute();
}
```

问题	后果
`total_price` 用前端传来的值	总价可以随意改
商品 `price`、`name` 用前端传来的值	单价和商品名可以随意改
不查商品是否存在	商品 id 为 0 或不存在也能下单（已知风险第 3 条）
不检查库存是否足够	库存不足也能下单，扣减时只是被截到 0

根本原因：后端把前端当成可信的。前端显示的价格只是给用户看的，任何人都能用抓包工具或 Postman 绕过页面直接改请求。

修复方案

原则：前端只告诉后端"买哪个商品、买几个"，单价、商品名和总价全部由后端从数据库查出来再计算，前端传来的 `price`、`name`、`total_price` 一律忽略。

改动	作用
按商品 id 查 products 表的名字、价格、库存	价格以数据库为准
商品不存在 → 返回 Product not found	同时修复已知风险第 3 条
数量小于等于 0 → 返回 Invalid item	防止负数数量把总价算成负数
库存不足 → 返回 Insufficient stock	防止超卖
先校验全部商品，再写数据库	任何一个商品不合法，就一条记录都不写
返回里带上后端算出的 `total_price`	前端和测试都能核对

修改方法：打开 `api/orders.php`，找到 `if ($method === 'POST' && $action === '') {` 这一段，从这一行开始，一直到 `// POST ?action=update_status – admin` 那行注释的上方，整段替换为下面的代码。修改前先把原文件复制一份备份。

```
if ($method === 'POST' && $action === '') {
    $data      = json_decode(file_get_contents('php://input'), true);
    $items     = $data['items']            ?? [];
    $payment   = $data['payment_method']   ?? 'card';
    $shipName  = $data['shipping_name']    ?? '';
    $shipPhone = $data['shipping_phone']   ?? '';
    $shipAddr  = $data['shipping_address'] ?? '';

    $rawUserId = $data['user_id'] ?? null;
    if ($rawUserId !== null) {
        $userId = (int)$rawUserId;
        if ($userId <= 0) {
            echo json_encode(['success' => false, 'message' => 'Invalid user_id']);
            exit;
        }
    } else {
        $userId = null;
    }

    if (!$shipName || !$shipPhone || !$shipAddr || empty($items) || !is_array($items)) {
        echo json_encode(['success' => false, 'message' => 'Missing required fields']);
        exit;
    }

    $priceStmt  = $db->prepare("SELECT name, price, stock FROM products WHERE id = ?");
    $orderItems = [];
    $total      = 0;

    foreach ($items as $item) {
        $pid = (int)($item['id']       ?? 0);
        $qty = (int)($item['quantity'] ?? 0);
        if ($pid <= 0 || $qty <= 0) {
            echo json_encode(['success' => false, 'message' => 'Invalid item']);
            exit;
        }

        $priceStmt->bind_param("i", $pid);
        $priceStmt->execute();
        $product = $priceStmt->get_result()->fetch_assoc();

        if (!$product) {
            echo json_encode(['success' => false, 'message' => 'Product not found']);
            exit;
        }
        if ((int)$product['stock'] < $qty) {
            echo json_encode(['success' => false, 'message' => 'Insufficient stock']);
            exit;
        }

        $price  = (float)$product['price'];
        $total += $price * $qty;
        $orderItems[] = ['id' => $pid, 'name' => $product['name'], 'quantity' => $qty,
'price' => $price];
    }
    $priceStmt->close();
    $total = round($total, 2);

    $tracking = 'TRK' . strtoupper(substr(str_shuffle('ABCDEFGHJKLMNPQRSTUVWXYZ23456789'),
0, 9));
```

```
    $stmt = $db->prepare(
        "INSERT INTO orders (user_id, total_price, payment_method, shipping_name,
shipping_phone, shipping_address, tracking_number)
         VALUES (?, ?, ?, ?, ?, ?, ?)"
    );
    $stmt->bind_param("idssssss", $userId, $total, $payment, $shipName, $shipPhone,
$shipAddr, $tracking);

    if (!$stmt->execute()) {
        echo json_encode(['success' => false, 'message' => 'Failed to create order']);
        $stmt->close();
        exit;
    }
    $orderId = $db->insert_id;
    $stmt->close();

    $itemStmt = $db->prepare(
        "INSERT INTO order_items (order_id, product_id, product_name, quantity, price)
VALUES (?, ?, ?, ?, ?)"
    );
    $stockStmt = $db->prepare("UPDATE products SET stock = stock - ? WHERE id = ?");
    foreach ($orderItems as $oi) {
        $itemStmt->bind_param("iisid", $orderId, $oi['id'], $oi['name'], $oi['quantity'],
$oi['price']);
        $itemStmt->execute();
        $stockStmt->bind_param("ii", $oi['quantity'], $oi['id']);
        $stockStmt->execute();
    }
    $itemStmt->close();
    $stockStmt->close();

    echo json_encode([
        'success'     => true,
        'order_id'    => $orderId,
        'tracking'    => $tracking,
        'total_price' => $total,
        'message'     => 'Order created successfully!'
    ]);
```

前端 `app.js` 不用改：它照样会把价格发过来，只是后端不再使用。修好后，同样用 Fiddler 把价格改成 0.01，订单金额也会是 $29,998。

比原来多的一处小改进：订单写入失败时不再把数据库错误 `$stmt->error` 返回给前端，避免暴露内部信息，和 BUG-012 的修复思路一致。

修复后的回归测试

修复后先做确认测试（这个缺陷修好没有），再做回归测试（下单的其他功能有没有被改坏）。2026-10-21 执行，全部通过。

编号	报告用例号	用例	预期结果	结果
R1	TC52	重复本缺陷的篡改步骤（Fiddler 改 price、total_price 为 0.01）	订单金额 $29,998，商品行 $14,999 × 2	通过
R2	TC53	网页正常下单 1 件	下单成功，金额 $14,999	通过
R3	TC54	Postman 下单，商品 id 9999	`Product not found`，不生成订单	通过
R4	TC55、TC56	Postman 下单，quantity 为 0 和 -1	`Invalid item`，不生成订单	通过
R5	TC57	Postman 下单，quantity 99999	`Insufficient stock`，库存不变	通过
R6	TC58	R2 和 R1 后查询库存	库存减少 3	通过
R7	TC50、TC51	Orders 页面、管理员后台	正常显示，金额正确	通过
R8	TC59	pytest 自动化用例	全部 PASSED	通过

把 R1 写成 pytest 自动化用例，放进 `test_orders.py`，以后每次改代码都能自动检查价格不会被篡改。它会真实生成一笔订单并扣减库存，只在本地测试环境运行。

```
def test_order_price_cannot_be_tampered(customer, base_url):
    product = requests.get(f"{base_url}/products.php", params={"id": 2}).json()
["products"][0]
    r = customer.post(f"{base_url}/orders.php", json={
        "items": [{"id": 2, "name": product["name"], "price": 0.01, "quantity": 2}],
        "total_price": 0.01,
        "payment_method": "alipay",
        "shipping_name": "Test User",
        "shipping_phone": "0123456789",
        "shipping_address": "Test Address",
        "user_id": customer.user_id
    })
    body = r.json()
    assert body["success"] is True
    order = customer.get(f"{base_url}/orders.php", params={"id": body["order_id"]}).json()
["orders"][0]
    assert float(order["total_price"]) == float(product["price"]) * 2
```

修复前运行这条用例会失败（实际 0.01），修复后通过。这正好用来证明：用例能发现这个缺陷，修复也真的生效了。

测试中遇到的问题与收尾

测试中遇到两个问题，都是工具使用上的，不是系统缺陷：

问题	原因	解决
设置断点后下单，没有被拦截	过滤器 Hosts 只写了 `localhost`，而请求的 Host 是 `localhost.fiddler`，请求被隐藏	Hosts 改为 `localhost; localhost.fiddler`
订单页面一直是空的，每次刷新都要回 Fiddler 放行	`bpu orders.php` 会拦截所有地址含 orders.php 的请求，包括查询订单列表；断点没有取消	QuickExec 输入 `bpu` 回车取消，点 Go 放行全部；还不行就重启 Fiddler

测试数据清理：订单 #24 是篡改产生的脏数据，截图留证后可以删除，并恢复被扣减的 2 件库存。在 phpMyAdmin 执行前，先用 SELECT 确认是这一笔：

```
SELECT * FROM orders WHERE id = 24;
DELETE FROM order_items WHERE order_id = 24;
DELETE FROM orders WHERE id = 24;
UPDATE products SET stock = stock + 2 WHERE id = 2;
```

相关风险：同一个请求里的 `"user_id":17` 也是前端传的，后端没有和登录用户核对，理论上能把订单记到别人名下（已知风险第 2 条的另一半）。可以用同样的断点方法验证：把 user_id 改成别的用户 id 再放行。修复思路相同：已登录时用 session 里的用户 id，不用请求里的。